

组织成了一个存储器层次结构，如图 1-9 所示。在这个层次结构中，从上至下，设备的访问速度越来越慢、容量越来越大，并且每字节的造价也越来越便宜。寄存器文件在层次结构中位于最顶部，也就是第 0 级或记为 L0。这里我们展示的是三层高速缓存 L1 到 L3，占据存储器层次结构的第 1 层到第 3 层。主存在第 4 层，以此类推。



图 1-9 一个存储器层次结构的示例

存储器层次结构的主要思想是上一层的存储器作为低一层存储器的高速缓存。因此，寄存器文件就是 L1 的高速缓存，L1 是 L2 的高速缓存，L2 是 L3 的高速缓存，L3 是主存的高速缓存，而主存又是磁盘的高速缓存。在某些具有分布式文件系统的网络系统中，本地磁盘就是存储在其他系统中磁盘上的数据的高速缓存。

正如可以运用不同的高速缓存的知识来提高程序性能一样，程序员同样可以利用对整个存储器层次结构的理解来提高程序性能。第 6 章将更详细地讨论这个问题。

1.7 操作系统管理硬件

让我们回到 hello 程序的例子。当 shell 加载和运行 hello 程序时，以及 hello 程序输出自己的消息时，shell 和 hello 程序都没有直接访问键盘、显示器、磁盘或者主存。取而代之的是，它们依靠操作系统提供的服务。我们可以把操作系统看成是应用程序和硬件之间插入的一层软件，如图 1-10 所示。所有应用程序对硬件的操作尝试都必须通过操作系统。

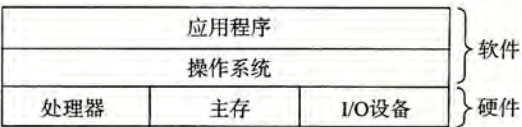


图 1-10 计算机系统的分层视图

操作系统有两个基本功能：(1) 防止硬件被失控的应用程序滥用；(2) 向应用程序提供简单一致的机制来控制复杂而又通常大不相同的低级硬件设备。操作系统通过几个基本的抽象概念(进程、虚拟内存和文件)来实现这两个功能。如图 1-11 所示，文件是对

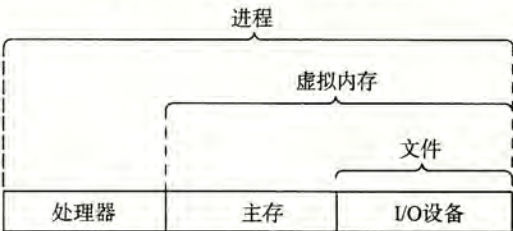


图 1-11 操作系统提供的抽象表示



I/O 设备的抽象表示, 虚拟内存是对主存和磁盘 I/O 设备的抽象表示, 进程则是对处理器、主存和 I/O 设备的抽象表示。我们将依次讨论每种抽象表示。

#### 旁注 Unix、Posix 和标准 Unix 规范

20 世纪 60 年代是大型、复杂操作系统盛行的年代, 比如 IBM 的 OS/360 和 Honeywell 的 Multics 系统。OS/360 是历史上最成功的软件项目之一, 而 Multics 虽然持续存在了多年, 却从来没有被广泛应用过。贝尔实验室曾经是 Multics 项目的最初参与者, 但是因为考虑到该项目的复杂性和缺乏进展而于 1969 年退出。鉴于 Multics 项目不愉快的经历, 一群贝尔实验室的研究人员——Ken Thompson、Dennis Ritchie、Doug McIlroy 和 Joe Ossanna, 从 1969 年开始在 DEC PDP-7 计算机上完全用机器语言编写了一个简单得多的操作系统。这个新系统中的很多思想, 比如层次文件系统、作为用户级进程的 shell 概念, 都是来自于 Multics, 只不过在一个更小、更简单的程序包里实现。1970 年, Brian Kernighan 给新系统命名为“Unix”, 这也是一个双关语, 暗指“Multics”的复杂性。1973 年用 C 重新编写其内核, 1974 年, Unix 开始正式对外发布[93]。

贝尔实验室以慷慨的条件向学校提供源代码, 所以 Unix 在大专院校里获得了很多支持并得以持续发展。最有影响的工作发生在 20 世纪 70 年代晚期到 80 年代早期, 在美国加州大学伯克利分校, 研究人员在一系列发布版本中增加了虚拟内存和 Internet 协议, 称为 Unix 4. xBSD(Berkeley Software Distribution)。与此同时, 贝尔实验室也在发布自己的版本, 称为 System V Unix。其他厂商的版本, 比如 Sun Microsystems 的 Solaris 系统, 则是从这些原始的 BSD 和 System V 版本中衍生而来。

20 世纪 80 年代中期, Unix 厂商试图通过加入新的、往往不兼容的特性来使它们的程序与众不同, 麻烦也就随之而来了。为了阻止这种趋势, IEEE(电气和电子工程师协会)开始努力标准化 Unix 的开发, 后来由 Richard Stallman 命名为“Posix”。结果就得到了一系列的标准, 称作 Posix 标准。这套标准涵盖了很多方面, 比如 Unix 系统调用的 C 语言接口、shell 程序和工具、线程及网络编程。最近, 一个被称为“标准 Unix 规范”的独立标准化工作已经与 Posix 一起创建了统一的 Unix 系统标准。这些标准化工作的结果是 Unix 版本之间的差异已经基本消失。

### 1.7.1 进程

像 hello 这样的程序在现代系统上运行时, 操作系统会提供一种假象, 就好像系统上只有这个程序在运行。程序看上去是独占地使用处理器、主存和 I/O 设备。处理器看上去就像在不间断地一条接一条地执行程序中的指令, 即该程序的代码和数据是系统内存中唯一的对象。这些假象是通过进程的概念来实现的, 进程是计算机科学中最重要和最成功的概念之一。

进程是操作系统对一个正在运行的程序的一种抽象。在一个系统上可以同时运行多个进程, 而每个进程都好像在独占地使用硬件。而并发运行, 则是说一个进程的指令和另一个进程的指令是交错执行的。在大多数系统中, 需要运行的进程数是多于可以运行它们的 CPU 个数的。传统系统在一个时刻只能执行一个程序, 而先进的多核处理器同时能够执行多个程序。无论是在单核还是多核系统中, 一个 CPU 看上去都像是在并发地执行多个进程, 这是通过处理器在进程间切换来实现的。操作系统实现这种交错执行的机制称为上下文切换。为了简化讨论, 我们只考虑包含一个 CPU 的单处理器系统的情况。我们会在 1.9.2 节中讨论多处理器系统。



操作系统保持跟踪进程运行所需的所有状态信息。这种状态，也就是上下文，包括许多信息，比如 PC 和寄存器文件的当前值，以及主存的内容。在任何一个时刻，单处理器系统都只能执行一个进程的代码。当操作系统决定要把控制权从当前进程转移到某个新进程时，就会进行上下文切换，即保存当前进程的上下文、恢复新进程的上下文，然后将控制权传递到新进程。新进程就会从它上次停止的地方开始。图 1-12 展示了示例 hello 程序运行场景的基本理念。

示例场景中有两个并发的进程：shell 进程和 hello 进程。最开始，只有 shell 进程在运行，即等待命令行上的输入。当我们让它运行 hello 程序时，shell 通过调用一个专门的函数，即系统调用，来执行我们的请求，系统调用会将控制权传递给操作系统。操作系统保存 shell 进程的上下文，创建一个新的 hello 进程及其上下文，然后将控制权传给新的 hello 进程。hello 进程终止后，操作系统恢复 shell 进程的上下文，并将控制权传回给它，shell 进程会继续等待下一个命令行输入。

如图 1-12 所示，从一个进程到另一个进程的转换是由操作系统内核(kernel)管理的。内核是操作系统代码常驻主存的部分。当应用程序需要操作系统的某些操作时，比如读写文件，它就执行一条特殊的系统调用(system call)指令，将控制权传递给内核。然后内核执行被请求的操作并返回应用程序。注意，内核不是一个独立的进程。相反，它是系统管理全部进程所用代码和数据结构的集合。

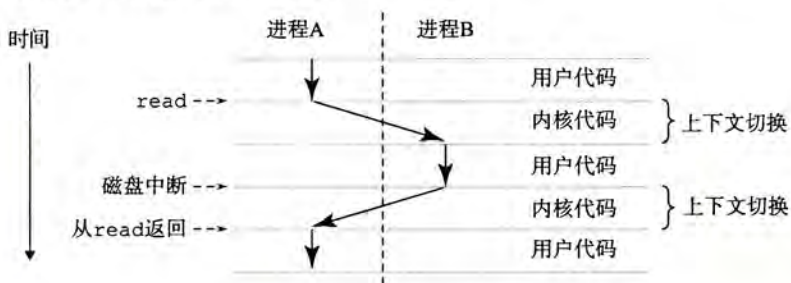


图 1-12 进程的上下文切换

实现进程这个抽象概念需要低级硬件和操作系统软件之间的紧密合作。我们将在第 8 章中揭示这项工作的原理，以及应用程序是如何创建和控制它们的进程的。

### 1.7.2 线程

尽管通常我们认为一个进程只有单一的控制流，但是在现代系统中，一个进程实际上可以由多个称为线程的执行单元组成，每个线程都运行在进程的上下文中，并共享同样的代码和全局数据。由于网络服务器中对并行处理的需求，线程成为越来越重要的编程模型，因为多线程之间比多进程之间更容易共享数据，也因为线程一般来说都比进程更高效。当有多处理器可用的时候，多线程也是一种使得程序可以运行得更快的方法，我们将在 1.9.2 节中讨论这个问题。在第 12 章中，你将学习并发的基本概念，包括如何写线程化的程序。

### 1.7.3 虚拟内存

虚拟内存是一个抽象概念，它为每个进程提供了一个假象，即每个进程都在独占地使用主存。每个进程看到的内存都是一致的，称为虚拟地址空间。图 1-13 所示的是 Linux 进程的

虚拟地址空间(其他 Unix 系统的设计也与此类似)。在 Linux 中,地址空间最上面的区域是保留给操作系统中的代码和数据的,这对所有进程来说都是一样。地址空间的底部区域存放用户进程定义的代码和数据。请注意,图中的地址是从下往上增大的。

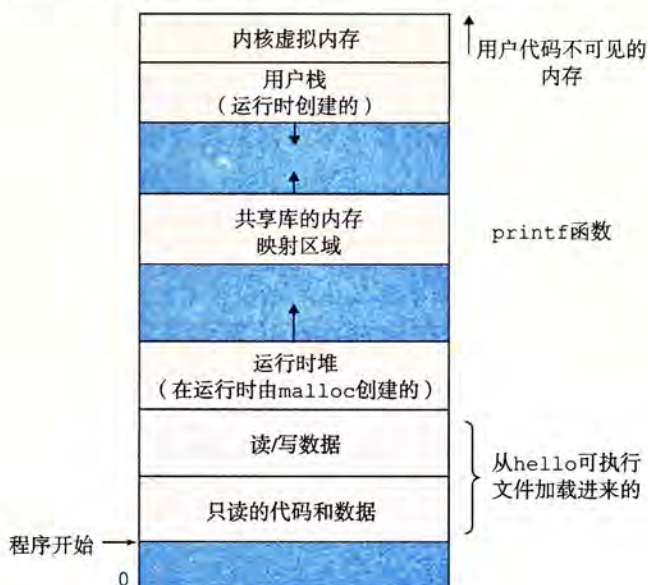


图 1-13 进程的虚拟地址空间

每个进程看到的虚拟地址空间由大量准确定义的区构成,每个区都有专门的功能。在本书的后续章节你将学到更多有关这些区的知识,但是先简单了解每一个区是非常有益的。我们从最低的地址开始,逐步向上介绍。

- 程序代码和数据。对所有的进程来说,代码是从同一固定地址开始,紧接着的是和 C 全局变量相对应的数据位置。代码和数据区是直接按照可执行目标文件的内容初始化的,在示例中就是可执行文件 `hello`。在第 7 章我们研究链接和加载时,你会学习更多有关地址空间的内容。
- 堆。代码和数据区后紧接着的是运行时堆。代码和数据区在进程一开始运行时就被指定了大小,与此不同,当调用像 `malloc` 和 `free` 这样的 C 标准库函数时,堆可以在运行时动态地扩展和收缩。在第 9 章学习管理虚拟内存时,我们将更详细地研究堆。
- 共享库。大约在地址空间的中间部分是一块用来存放像 C 标准库和数学库这样的共享库的代码和数据的区域。共享库的概念非常强大,也相当难懂。在第 7 章介绍动态链接时,将学习共享库是如何工作的。
- 栈。位于用户虚拟地址空间顶部的是用户栈,编译器用它来实现函数调用。和堆一样,用户栈在程序执行期间可以动态地扩展和收缩。特别地,每次我们调用一个函数时,栈就会增长;从一个函数返回时,栈就会收缩。在第 3 章中将学习编译器是如何使用栈的。
- 内核虚拟内存。地址空间顶部的区域是为内核保留的。不允许应用程序读写这个区域的内容或者直接调用内核代码定义的函数。相反,它们必须调用内核来执行这些操作。



虚拟内存的运作需要硬件和操作系统软件之间精密复杂的交互，包括对处理器生成的每个地址的硬件翻译。基本思想是把一个进程虚拟内存的内容存储在磁盘上，然后用主存作为磁盘的高速缓存。第9章将解释它如何工作，以及为什么对现代系统的运行如此重要。

#### 1.7.4 文件

文件就是字节序列，仅此而已。每个 I/O 设备，包括磁盘、键盘、显示器，甚至网络，都可以看成是文件。系统中的所有输入输出都是通过使用一小组称为 Unix I/O 的系统函数调用读写文件来实现的。

文件这个简单而精致的概念是非常强大的，因为它向应用程序提供了一个统一的视图，来看待系统中可能含有的所有各式各样的 I/O 设备。例如，处理磁盘文件内容的应用程序可以非常幸福，因为他们无须了解具体的磁盘技术。进一步说，同一个程序可以在使用不同磁盘技术的不同系统上运行。你将在第10章中学习 Unix I/O。

#### 旁注 Linux 项目

1991年8月，芬兰研究生 Linus Torvalds 谨慎地发布了一个新的类 Unix 的操作系统内核，内容如下。

来自：torvalds@klaava.Helsinki.FI(Linus Benedict Torvalds)

新闻组：comp.os.minix

主题：在 minix 中你最想看到什么？

摘要：关于我的新操作系统的小调查

时间：1991年8月25日 20:57:08 GMT

每个使用 minix 的朋友，你们好。

我正在做一个(免费的)用在 386(486)AT 上的操作系统(只是业余爱好，它不会像 GNU 那样庞大和专业)。这个想法自4月份就开始酝酿，现在快要完成了。我希望得到各位对 minix 的任何反馈意见，因为我的操作系统在某些方面与它相类似(其中包括相同的文件系统的物理设计(因为某些实际的原因))。

我现在已经移植了 bash(1.08)和 gcc(1.40)，并且看上去能运行。这意味着我需要几个月的时间来让它变得更实用一些，并且，我想要知道大多数人想要什么特性。欢迎任何建议，但是我无法保证我能实现它们。:-)

Linus (torvalds@kruuna.helsinki.fi)

就像 Torvalds 所说的，他创建 Linux 的起点是 Minix，由 Andrew S. Tanenbaum 出于教育目的开发的一个操作系统[113]。

接下来，如他们所说，这就成了历史。Linux 逐渐发展成为一个技术和文化现象。通过和 GNU 项目的力量结合，Linux 项目发展成了一个完整的、符合 Posix 标准的 Unix 操作系统的版本，包括内核和所有支撑的基础设施。从手持设备到大型计算机，Linux 在范围如此广泛的计算机上得到了应用。IBM 的一个工作组甚至把 Linux 移植到了一块腕表中！

### 1.8 系统之间利用网络通信

系统漫游至此，我们一直是把系统视为一个孤立的硬件和软件的集合体。实际上，现